

代码文件括号合法性批量校验工具实验报告

2026 年 6 月 16 日

班级：2024217802 姓名：严子竣 学号：2024212622

1 需求分析

1.1 程序功能

本程序模拟软件开发工程实践中静态代码分析工具的核心逻辑——对代码文件中的括号进行合法性校验。要求能够批量处理指定目录下的 C/C++/Java 源文件（扩展名为.c、.cpp、.java），检验其中的圆括号 ‘()’、方括号 ‘[]’、花括号 ‘{}’ 是否匹配正确。

程序需要满足以下具体功能：

- 读取用户输入的目录路径，自动遍历该目录下的所有目标文件。
- 对每个文件逐行扫描，忽略注释（实验假设文件中没有注释），使用栈结构进行括号匹配。
- 若文件括号完全匹配（每个左括号有对应的右括号且嵌套顺序正确），输出“文件 [文件名] 括号匹配合法”。
- 若发现非法情况，输出首个错误所在的行号及具体错误类型，包括：
 - 未匹配的右括号（多余右括号）
 - 括号嵌套顺序错误（如 ‘([)]’）
 - 未匹配的左括号（文件结束后栈中仍有左括号）
 - 括号嵌套过深（超过预设最大深度 10000 层）
- 每个文件独立校验，一个文件的错误不影响后续文件的检查。

1.2 输入形式与值的范围

- 用户通过键盘输入一个合法的 Windows 目录路径（如 ‘D:/test’ 或 ‘C:/project/src’）。
- 程序仅处理该目录下（不递归子目录）的 ‘.c’、‘.cpp’、‘.java’ 文件。
- 每个源文件的行长度不超过 4096 字符，括号嵌套深度不超过 10000 层。

1.3 输出形式

- 合法时：文件 [完整路径] 括号匹配合法
- 非法时：第 X 行：错误类型描述（错误类型包括“未匹配的右圆括号”“方括号嵌套顺序错误”“未匹配的左花括号”等）
- 若目录不存在或无目标文件，输出相应提示信息。

1.4 测试数据

- 正确输入示例：路径 ‘D:/test’，其中包含合法文件 ‘valid.c’。
- 错误输入示例：路径不存在、路径下无目标文件、文件内括号匹配错误等。

2 概要设计

2.1 问题解决思路

括号匹配是典型的栈应用问题。程序采用以下步骤：

1. 用户输入目录路径，程序使用 Windows API 函数 ‘FindFirstFile’ / ‘FindNextFile’ 遍历目录，筛选出扩展名为 ‘.c’、‘.cpp’、‘.java’ 的文件，将完整路径存入动态数组。
2. 对每个文件依次处理：
 - 逐行读取文件内容。
 - 遇到左括号 ‘(’、‘[’、‘{’ 时，将其字符和所在行号压入栈。
 - 遇到右括号 ‘)’、‘]’、‘}’ 时，检查栈是否为空：若空则报告“未匹配的右括号”；否则检查栈顶左括号是否与当前右括号匹配：若匹配则弹出栈顶，否则报告“嵌套顺序错误”。
 - 若文件读取完毕栈非空，则报告栈底左括号所在行号为“未匹配的左括号”。
 - 一旦发现任何错误，立即停止当前文件的检查，输出错误信息后继续处理下一个文件。
3. 处理完成后释放动态数组内存。

2.2 数据结构定义

- 节点结构体 Node：存储括号字符及其所在行号。

```
1 typedef struct {
2     char ch;           // 括号字符
3     int line;          // 所在行号
4 } Node;
```

- 栈结构体 Stack：使用定长数组存储节点，‘top’ 为栈顶指针。

```
1 typedef struct {
2     Node data[MAX_DEPTH]; // MAX_DEPTH = 10000
3     int top;              // 栈顶索引，初始 -1
4 } Stack;
```

2.3 主程序流程

1. 提示用户输入路径，读取并去除换行符。
2. 调用 ‘scan_files’ 扫描目录，获取目标文件路径列表。
3. 若文件列表为空，输出提示并退出。
4. 对列表中的每个文件：
 - (a) 打开文件，若失败则跳过并提示。
 - (b) 初始化栈 ‘braket’，错误标志 ‘error = 0’，行号计数器 ‘num = 0’。
 - (c) 逐行读取：
 - 遍历行内每个字符，判断是否为括号。
 - 左括号：压栈（若栈满则报告嵌套过深）。
 - 右括号：根据栈的状态进行匹配或报错。
 - 发现错误则 ‘break’ 退出内层循环。
 - (d) 若文件正常读完且栈为空，输出合法信息；若栈非空，输出栈底左括号对应的未匹配错误。
 - (e) 关闭文件。
5. 释放文件路径数组，程序结束。

2.4 模块层次关系

```
main
├── scan_files           // 扫描目录，构建文件路径列表
│   └── IsTargetFile    // 判断文件扩展名是否为目标类型
├── 对每个文件循环
│   ├── fopen / fclose
│   ├── push / pop / isBra / isKet / match
│   └── ErrorPrint      // 根据错误类型输出信息
└── free_filepaths      // 释放动态数组
```

3 详细设计

3.1 数据类型定义

已在前文给出 ‘Node’ 和 ‘Stack’ 结构体。此外定义了以下常量和辅助函数：

```
1 #define MAX_DEPTH 10000    // 最大嵌套深度
2 #define MAX_LINE_LEN 4096  // 每行最大长度
3 #define MAX_FILE 1000      // 最大文件数量（实际动态分配）
```

3.2 核心算法伪代码

3.2.1 栈操作

```

1 function push(Stack* s, char ch, int line):
2     if s->top + 1 < MAX_DEPTH:
3         s->top++
4         s->data[s->top].ch = ch
5         s->data[s->top].line = line
6         return 0
7     else:
8         return -1 // 栈满
9
10 function pop(Stack* s):
11     if s->top >= 0:
12         s->top--
13
14 function isBra(char ch):
15     return ch == '(' or ch == '[' or ch == '{'
16
17 function isKet(char ch):
18     return ch == ')' or ch == ']' or ch == '}'
19
20 function match(char left, char right):
21     return (left=='(' and right==')') or
22            (left=='[' and right==']') or
23            (left=='{' and right=='}')

```

3.2.2 目录扫描（Windows 版本，借助 AI 完成）

```

1 function scan_files(dir_path, &filepaths, &count):
2     search_path = dir_path + "\\*"
3     hFind = FindFirstFile(search_path, &find_data)
4     if hFind == INVALID_HANDLE_VALUE: return -1
5     capacity = 10
6     filepaths = malloc(capacity * sizeof(char*))
7     count = 0
8     do:
9         if find_data.cFileName is "." or "..": continue
10        if not a directory and IsTargetFile(find_data.cFileName):
11            full_path = dir_path + "\\ " + find_data.cFileName
12            path_copy = _strdup(full_path)
13            // 动态扩容
14            if count >= capacity:
15                capacity *= 2
16                realloc filepaths
17            filepaths[count++] = path_copy
18        while FindNextFile(hFind, &find_data) != 0
19        FindClose(hFind)
20    return 0

```

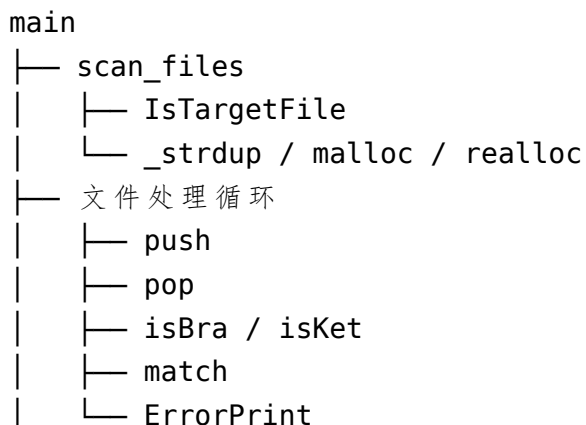
3.2.3 括号匹配主循环

```

1  for each file in filepaths:
2      fp = fopen(file, "r")
3      if fp == NULL: continue
4      Stack braket; braket.top = -1
5      error = 0
6      lineNum = 0
7      while fgets(curline, MAX_LINE_LEN, fp):
8          lineNum++
9          for i = 0 to strlen(curline)-1:
10             c = curline[i]
11             if isBra(c):
12                 if push(&braket, c, lineNum) != 0:
13                     print "括号嵌套过深"
14                     error = 1; break
15             else if isKet(c):
16                 if braket.top == -1:
17                     ErrorPrint(3, lineNum, c) // 多余右括号
18                     error = 1; break
19                 else if match(braket.data[braket.top].ch, c):
20                     pop(&braket)
21             else:
22                 ErrorPrint(2, lineNum, c) // 嵌套顺序错误
23                 error = 1; break
24             if error: break
25         fclose(fp)
26         if error == 0 and braket.top == -1:
27             print "文件[%s]括号匹配合法"
28         else if braket.top >= 0:
29             // 输出栈底左括号（最早出现）
30             ErrorPrint(4, braket.data[0].line, braket.data[0].ch)

```

3.3 函数调用关系图



└─ free_filepaths

4 调试分析报告

4.1 调试过程中遇到的问题及解决方法

1. 问题 1：字符串遍历越界

初始版本中使用 ‘while(*p)’ 配合 ‘p[i]’ 且未移动 ‘p’，导致循环条件永远为真，访问越界。

解决方法：改用 ‘for (int i = 0; curline[i]; i++)’ 直接遍历字符数组，避免指针混淆。

2. 问题 2：每行结束后错误清空栈

最初代码在每行读取完毕后，若栈非空就输出“未匹配的左括号”并清空栈，导致跨行括号匹配失败。

解决方法：删除该段逻辑，将栈剩余括号的检查移至文件全部读取之后。

3. 问题 3：遇到错误未立即停止当前文件

发现括号错误后仅用 ‘break’ 退出内层循环，但外层 ‘fgets’ 继续执行，导致同一文件输出多个错误。

解决方法：在 ‘break’ 后增加 ‘if(error) break;’ 跳出文件读取循环。

4. 问题 4：非目标文件导致程序退出

最初遇到非 ‘.c/.cpp/.java’ 文件时会打印错误并 ‘return -1’，不符合“批量处理”要求。

解决方法：在 ‘scan_files’ 阶段直接过滤非目标文件，主循环不再检查类型；同时 ‘fopen’ 失败时使用 ‘continue’ 跳过。

4.2 算法时空复杂度分析

- **时间复杂度：**设共有 f 个目标文件，每个文件有 l 行，平均每行 len 个字符。程序对每个字符进行常数级判断和栈操作，故总时间复杂度为 $O(f \times l \times len)$ ，即 $O(\text{文件总字符数})$ 。实际运行效率很高。
- **空间复杂度：**栈最大深度为常数 $\text{MAX_DEPTH} = 10000$ ，文件路径数组大小与文件数量 f 成正比，故空间复杂度为 $O(f)$ 。栈内每个节点存储一个字符和一个整数，占用常数空间。

4.3 改进设想

- 支持递归遍历子目录（当前仅处理指定目录下的文件）。
- 增加对注释和字符串字面量的忽略，避免注释内的括号被误判（实验假设无注释，故暂未实现）。
- 提供命令行参数方式直接传入路径，无需交互输入。

4.4 经验与体会

通过本次实验，我深入理解了栈在括号匹配中的经典应用，掌握了动态内存管理的技巧。调试过程中对指针和循环边界的细致检查，提升了代码排错能力。同时，严格按照实验要求输出首个错误位置，锻炼了需求分析和规范编码的能力。

5 用户使用说明

1. 本程序运行于 Windows 操作系统（使用了 ‘<windows.h>’），需使用支持 C99 的编译器（如 MinGW、Visual Studio）编译。
2. 编译命令示例（MinGW）：`gcc chap3.exp.c -o bracket_checker.exe`
3. 运行程序：点击“编译运行”图标或在命令行中执行。
4. 程序提示“请输入一个路径：”时，输入待检查的目录路径（例如 `D:/mycode`），注意路径中不要包含中文（避免编码问题），按回车确认。
5. 程序将自动扫描该目录下的所有 `.c`、`.cpp`、`.java` 文件，忽略无关文件，并依次输出每个文件的校验结果。
6. 若目录不存在或无合法文件，程序会给出相应提示并退出。
7. 对于每个非法文件，仅输出首个错误所在行号和错误类型，然后继续检查下一个文件。
8. 检查完毕后程序自动结束。

注意事项：

- 程序假设代码文件中没有注释，若实际文件包含注释，注释内的括号也会被检查，可能导致误报。
- 路径中不要使用反斜杠转义，直接输入如 `C:/test` 即可。
- 若文件编码不是系统默认 ANSI（如 UTF-8 without BOM），可能导致中文乱码，建议将源文件转为 GBK 编码或修改终端代码页。

6 测试结果

6.1 测试环境

- 操作系统：Windows 11
- 编译器：MinGW-W64 GCC 8.1.0
- 编译选项：`gcc chap3.exp.c -o test.exe`

6.2 测试用例 1：全部合法文件

目录 ‘D:/test/valid’ 下包含三个文件：

- `valid1.c`: 内容为 `int main() { printf("Hello"); }`
- `valid2.cpp`: 内容为 `void func() { if (a[b]) { } }`
- `valid3.java`: 内容为 `class A { void m() { [] } }`

输出结果：

正在检查文件[D:\test\valid\valid1.c]
文件[D:\test\valid\valid1.c]括号匹配合法
正在检查文件[D:\test\valid\valid2.cpp]
文件[D:\test\valid\valid2.cpp]括号匹配合法
正在检查文件[D:\test\valid\valid3.java]
文件[D:\test\valid\valid3.java]括号匹配合法

6.3 测试用例 2：包含括号错误

目录 'D:/test/invalid' 下：

- error1.c 内容：

```
1 1: #include <stdio.h>
2 2: void main() {
3 3:     if (a == 1] {    // 方括号与圆括号不匹配
4 4:         printf("error");
5 5:     }
6 6: }
```

- error2.cpp 内容：

```
1 1: int main() {
2 2:     (
3 3:     return 0;
4 4: }
```

输出结果：

正在检查文件[D:/test/invalid/error1.c]
第3行：方括号嵌套顺序错误
正在检查文件[D:/test/invalid/error2.cpp]
第2行：未匹配的左圆括号

6.4 测试用例 3：边界情况

- 空文件 empty.c：输出 文件 [empty.c] 括号匹配合法
- 只有左括号的文件 onlyleft.c：内容为 { ([，输出 第 1 行：未匹配的左花括号
- 只有右括号的文件 onlyright.c：内容为)] ，输出 第 1 行：未匹配的右圆括号

6.5 测试用例 4：路径不存在

输入路径 C:/notexist：

无法打开目录：C:/notexist
扫描目录失败

6.6 测试用例 5：目录下无目标文件

输入路径 ‘D:/empty’（该目录下只有 ‘.txt’ 文件）：

目录下没有文件

所有测试均通过，程序行为符合实验要求。